

CS466 Lab 2 -- Hardware, Development Tools and Blinking the LED using FreeRTOS

Due by Midnight Sunday 1/31/2026.

Use provided lab format on Canvas

Note: This is an individual lab, you are free to collaborate but every student must perform the lab and hand in a lab report. It will be critical that each student is able to develop to the target platform..

Overview:

Lab Preparation:

Review your Lab1 problem and solution.. Does it meet all the requirements in step 10 below?

Update (git pull) your class repository, be sure to commit any changes that you have first.

Take time to look at the CMakeLists.txt file. The pico and FreeRTOS code is buile implicitly can you see how it's pulled in?

Locate the FreeRTOS API documentation under 'API References' at https://www.freertos.org/Documentation/02-Kernel/07-Books-and-manual/01-RTOS_book and read about the xTaskCreate(), xSemaphoreTake() and xSemaphoreGiveFromISR() FreeRTOS functions in detail. Take special care to understand the second parameter in xSemaphoreTake() and what it's behavior will be if this value is 0 or greater.

Take a detailed look at the provided lab2.c program.

- o The main program sets up the LED hardware in the same manner.
- o The heartbeat() function will blink the LED at 1Hz at startup.
- o Look at main and notice how the example registers a new 'Task' and sets it to point to the heartbeat() method.
- o There is an interrupt handler in place to detect the SW1 press. What does pressing SW1 do?
- o Look how the interrupt routine is registered with the pico architecture.
- o Notice that we now have 'printf()' calls. How was that enabled?
- o Notice that there is commented out semaphore code and an example SW1 handler for some aid.

Take a detailed look at the config file FreeRTOSConfig.h

Take time to look at the CMakeLists.txt file.

Objectives:

There are three objectives of this lab.

1. To discover that the RTOS helps organize and make your code more independent and flexible.
2. To cause Interrupts and alter the corresponding interrupt handlers
3. To generate and log serial debug output using the psuedo-serial device in the USB service.

Lab Work:

1. Perform a 'git pull' on the class repo to get the required lab2 directory updates..

- Follow the instructions in the cs466_s24 README.md file to add the FreeRTOS environment to your directory Tree. Note that I have switched to the latest FreeRTOS-LTS. You can get the latest LTS using the command line;

wget <https://github.com/FreeRTOS/FreeRTOS-LTS/releases/download/202406.04-LTS/FreeRTOSv202406.04-LTS.zip>

The class CmakeLists.txt files expect the FreeRTOS-LTS top directory to be a peer of the pico and cs466_s26 directories. If you move it you will need to adjust the settings near the top of the CmakeLists.txt file.

- Verify that the ROOT dirs in the lab2 CmakeLists.txt files match your setup.
- Build and run the lab2 program and verify that the SW1 behavior matches the description in the lab2 see how SW1 (Which I have assigned to pard pin 22, GPIO17) alters its behavior.

LAB-Questions:

- Describe the program behavior
 - Is it consistent, discuss your result and probable causes.
- Setup a serial path so that we can display debugging information back on your development machine. When the program starts there is a call in main() called `stdio_init_all()` that will enable the pico functionality to receive serial print data over the USB cable. After the program is running it should show up as a tty device in `/dev`.
 - On Ubuntu you can run the command ``dmesg`` or ``sudo dmesg`` after the program starts to determine if the pico has registered itself as a serial device. On my workstation the interesting out (at the end of my ``dmesg`` output) looks like;

```
[961041.651472] usb 2-1.5.1.3: new full-speed USB device number 52 using ehci-pci
[961041.777784] usb 2-1.5.1.3: New USB device found, idVendor=2e8a, idProduct=000a, bcdDevice= 1.00
[961041.777789] usb 2-1.5.1.3: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[961041.777792] usb 2-1.5.1.3: Product: Pico
[961041.777794] usb 2-1.5.1.3: Manufacturer: Raspberry Pi
[961041.777796] usb 2-1.5.1.3: SerialNumber: E66118604B381828
[961041.780863] cdc_acm 2-1.5.1.3:1.0: ttyACM0: USB ACM device
```

- From the command output I see that my workstation has set the USB device up as `/dev/ttyACM0` but by default this device will only be available if you are running as root.
- Linux has a service ``udev`` that allows running configuration whenever devices are connected. Using the `idVendor=2e8a, idProduct=000a` information you can construct a udev file and register it with the udev process that will make the `/dev/ttyACM0` device available to users
- Create a file `/etc/udev/rules.d/95-pi-pico-udev.rules` that contains the following two lines (the second line is long and wraps below.. Don't wrap it in the udev file.);

```
# raspberry pi-pico
ATTRS{idVendor}=="2e8a", ATTRS{idProduct}=="000a", MODE="0666",
ENV{ID_MM_DEVICE_IGNORE}="1", ENV{ID_MM_PORT_IGNORE}="1"
```
- After creating the udev file you can reboot your Linux or it's much easier to just restart the udev service using the command;

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```
- Restart your pi-pico board and check that you have read-write access to `/dev/ttyACM0`. A `ls` command should return the following;

```
$ ls -l /dev/ttyACM0
crw-rw-rw- 1 root dialout 166, 0 Jan 24 17:58 /dev/ttyACM0
```

g. If the protections are not 666 (rw-rw-rw) for the device you have an issue somewhere.

6. use your serial program of choice to listen to /dev/ttyACM0. I use kermit but any will work. The serial port should be setup as 115200-n-8-1. I have put a kermit executable and setup file in the lab directory, these will probably only work on Ubuntu 20.04 but the source can be found and recompiled pretty easily for other platforms.

```
$ kermit ~/kermACM0
Connecting to /dev/ttyACM0, speed 115200
Escape character: Ctrl-\ (ASCII 28, FS): enabled
Type the escape character followed by C to get back,
or followed by ? to see other options.
-----
hb-tick: 125
hb-tick: 125
hb-tick: 125
...
```

7. Remove the code that changes the heartbeat frequency if SW1 pressed.
8. Add code to cause pressing SW1 (GPIO17) and SW2 (GPIO16) to fire the interrupt routine for you.
- SW1 is done for you but commented out, add setup and interrupt connection for SW2
 - Create two semaphores (variables) that are static global at the module level.
 - In main() before you create the threads initialize the semaphore objects to the result of xSemaphoreCreateBinary();
 - In the ISR, if SW1 is pressed give a semaphore (e.g. semSW1). If SW2 is pressed give a different semaphore (e.g. semSW2)

9. Add two new FreeRTOS tasks to the system for each of the other two LED behaviors, start simple. Make the two tasks have higher priority than the heartbeat task.

- Generally RTOS task implementations should not return.
- The task paradigm that we will always use as task structure is:
 - Appear to be a normal void function.
 - Perform some setup optional operations.
 - Start an infinite loop that will run forever.
 - Be sure to block on something or you will starve lower priority tasks.
 - Do something meaningful.

In my SW1 receive task I satisfy the above template requirements

```
void sw1_handler(void * notUsed)
{
    while (true)
    {
        xSemaphoreTake( _semBtn, portMAX_DELAY); // this will block;
        printf("sw1 Semaphore taken..\n");
        // do something useful here....;
    }
}
```

10. Using only interrupt detection of SW1 and SW2 press, modify the code so that:
- If no buttons are pressed, the green LED continuously blinks at 1.0Hz
 - Pressing SW1 will cause the led to flash 20 times at 15Hz
 - Pressing SW2 will cause the led to flash 10 times at 13Hz
 - If both buttons are pressed the LED should flash continuously at 5 Hz.
 - For single buttons, use only the button press as a trigger, the led should blink their required number of times no matter how long the button is pressed.

11. Notice that we FreeRTOS tasks use `vTaskDelay` in place of the non-RTOS `delay_ms/delay_us`.
LAB-Question: What is the minimum delay resolution that you can attain.
12. By now you have probably noticed that a single button press may cause multiple interrupt occurrences. Put code in the ISR that will add a 25ms guard time after a button press to ignore bounces that may occur when you press the button.
13. Extra Credit! If you don't press a button in 60 seconds, cause the LED to flicker at 20Hz for 1 second to remind the user that they are available.